Fullstack-Spring-Boot Projekt - Booktracker

1. **Neues Projekt anlegen**
    - Öffne IntelliJ und klicke auf New Project.
    - Wähle in der linken Liste Spring Initializr aus.
    - Einstellungen:
        1. Name: booktracker
        2. Location: Wähle einen Ordner auf deiner Festplatte.
        3. Language: Java
        4. Type: Maven
        5. Group: deine Domain, z.B. com.portfolio
        6. Artifact: booktracker
        7. JDK: Wähle dein JDK.
        8. Java: Wähle deine Version (>17).
        9. Packaging: Jar
    - Klicke auf Next.

2. **Dependencies (Abhängigkeiten) auswählen**

Kategorie	Name der Dependency	Zweck
Web	Spring Web	Erstellt die REST-Endpunkte und den Server
SQL	Spring Data JPA	Speichert die Daten in der DB
SQL	MySQL Driver	Verbindung zur DB
Developer Tools	Spring Boot Dev Tools	Auto-Restart bei Änderungen

Developer Tools	Lombok	Erstellt automatisch Getter/Setter/toString/Konstruktoren
I/O	Validation	Prüft Formulareingaben

3. **Wichtige Einstellung für Lombok in IntelliJ**

   Damit Lombok (für die automatischen Getter/Setter) in IntelliJ funktioniert, musst du eine Einstellung aktivieren:

   1. Gehe zu File -> Settings (bei Mac: IntelliJ IDEA -> Settings).
   2. Navigiere zu Build, Execution, Deployment -> Compiler -> Annotation Processors.
   3. Setze den Haken bei Enable annotation processing.
   4. Klicke auf OK.

4. **Orderstruktur finalisieren**
   - Erstelle in deinem java-Ordner ganz drin (unter src/main/java) folgende Ordner (Packages):

   1. Rechtsklick auf booktracker -> **New** -> **Package**: model
   2. Rechtsklick auf booktracker -> **New** -> **Package**: repository
   3. Rechtsklick auf booktracker -> **New** -> **Package**: service
   4. Rechtsklick auf booktracker -> **New** -> **Package**: controller

5. **Der erste Check**
   - Füge der Application Klasse folgendes hinzu: @SpringBootApplication(exclude = {DataSourceAutoConfiguration.class})
   - (Das deaktiviert die Datenbank-Suche vorübergehend, damit wir die App einmal starten können.)
   - App starten

6. Datenmodell erstellen

- Gehe in das Package model (das du vorhin erstellt hast).
- Erstelle eine neue Java-Klasse: Book.java.
- Kopiere den Code.

```java
package com.raphaeladohrmann.fullstack_demo.model;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data                // Generiert Getter, Setter, toString, equals/hashCode
@NoArgsConstructor   // Generiert leeren Konstruktor
@AllArgsConstructor  // Generiert Konstruktor mit allen Feldern
public class Book {
    private Long id;
    private String title;
    private String author;
    private String isbn;
    private int rating;      // 1-5 Sterne
    private String status;   // z.B. "Gelesen", "Geplant"

}
```

7. **Service erstellen (Logik-Zentrale)**
    - Gehe ins package service und erstelle eine Klasse BookService

```java
package com.raphaeladohrmann.booktracker.service;

import com.raphaeladohrmann.booktracker.model.Book;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.List;

@Service  no usages
public class BookService {

    // Liste als "temporärer Speicher"
    private List<Book> books = new ArrayList<>();  4 usages

    public BookService() {
        // initialer Datenbestand zum Testen
        books.add(new Book( id: 1L,  title: "Der Hobbit",  author: "J.R.R. Tolkien",  isbn: "978-3423214125",  rating: 5,  status: "Gelesen"));
        books.add(new Book( id: 2L,  title: "Clean Code",  author: "Robert C. Martin",  isbn: "978-0132350884",  rating: 4,  status: "Lesend"));
    }

    // Methode zur Ausgabe aller Bücher
    public List<Book> findAll() {  no usages
        return books;
    }

    // Methode zum Hinzufügen eines Buchs
    public void add(Book book) {
        books.add(book);
    }
}
```

8. **Die Schnittstelle (Controller)**
    - Hier verknüpfen wir unser Book-Model mit der Außenwelt – der Controller nutzt den Service, um die benötigten Daten zu liefern.
    - Erstelle im package controller eine BookController Klasse.

```java
package com.raphaeladohrmann.booktracker.controller;

import com.raphaeladohrmann.booktracker.model.Book;
import com.raphaeladohrmann.booktracker.service.BookService;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/books")
public class BookController {

    private final BookService bookService;  3 usages

    // Constructor Injection ->
    // Spring weiß: um einen BookController zu bauen, brauche ich einen BookService
    // -> sucht diesen und gibt ihn hinein
    public BookController(BookService bookService) {
        this.bookService = bookService;
    }

    // Abrufen aller Bücher
    @GetMapping
    public List<Book> getAllBooks() {
        return bookService.findAll();
    }

    // Ein neues Buch per JSON hinzufügen
    @PostMapping
    public String createBook(@RequestBody Book book) {
        bookService.add(book);
        return "Buch erfolgreich empfangen: " + book.getTitle();
    }
}
```

Was wir hier lernen:

1. **JSON-Mapping:** Spring Boot erkennt automatisch, dass ein eingehendes JSON (z.B. vom Frontend) genau in deine Book-Klasse passt, solange die Feldnamen (title, author, etc.) übereinstimmen.

2. **Stateless vs. Stateful:** Wenn sie die App neu starten, sind die neu hinzugefügten Bücher weg. Das ist der perfekte Moment um gleich eine DB anzubinden.
3. **HTTP-Methoden:** GET ist zum "Ansehen" (Browser reicht), POST ist zum "Senden" (Spezial-Tool nötig).

9. http-Requests testen

- Führe im Browser http://localhost:8080/api/books aus. Wir sollten die zwei Bücher sehen.
- Automatisierung:
  1. Ganz oben im Strukturbaum: New -> File -> book-requests.http
  2. Inhalt einfügen:

```
### 1. Alle Bücher abrufen (GET)
# Dies entspricht dem Aufruf im Browser.
GET http://localhost:8080/api/books
Accept: application/json

###

### 2. Ein neues Buch hinzufügen (POST)
# Hier schicken wir Daten zum Server.
# Wichtig: Der Content-Type sagt Spring, dass wir JSON senden.
POST http://localhost:8080/api/books
Content-Type: application/json

{
  "id": 3,
  "title": "Clean Architecture",
  "author": "Robert C. Martin",
  "isbn": "978-0134494166",
  "rating": 5,
  "status": "Geplant"
}
```

An dieser Stelle solltet ihr nochmal die Basics von JSON und REST wiederholen und euch ggf. nochmal mit dem Methoden vertraut machen.

10. Das Repository (Die Brücke zur DB)

- Zuerst brauchen wir ein Interface, dass die SQL-Befehle für uns schreibt. Im package repository erstellen wir das Interface BookRepository.

```java
package com.raphaeladohrmann.booktracker.repository;

import com.raphaeladohrmann.booktracker.model.Book;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

/*
 * JpaRepository bietet uns alle Standard-Operationen:
 * save(), findAll(), findById(), deleteById() usw.
 * Wir müssen hier keinen einzigen SQL-Befehl selbst schreiben,
 * außer wir brauchen individuelle Abfragen.
 */
@Repository  no usages
public interface BookRepository extends JpaRepository<Book, Long> {
}
```

11. Repo im Service aufrufen

- Um den Service mit dem Repo zu verbinden, schmeißen wir unsere ArrayList raus und setzten das Repo im Konstruktor.

```java
package com.raphaeladohrmann.booktracker.service;

import com.raphaeladohrmann.booktracker.model.Book;
import com.raphaeladohrmann.booktracker.repository.BookRepository;
import org.springframework.stereotype.Service;

import java.util.List;

@Service  3 usages
public class BookService {

    private final BookRepository bookRepository;  3 usages

    // Konstruktor-Injektion: Spring reicht das Repository hier rein
    public BookService(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    // Methode zur Ausgabe aller Bücher
    public List<Book> findAll() {  1 usage
        // Holt jetzt alle Einträge aus der echten Datenbank-Tabelle
        return bookRepository.findAll();
    }

    // Methode zum Hinzufügen eines Buchs
    public Book add(Book book) {
        // Speichert das Objekt dauerhaft in der Datenbank
        return bookRepository.save(book);
    }
}
```

12. **Unser Model zur Entity machen**
    - Damit Hibernate (das Tool hinter JPA) weiß, wie die DB Tabelle aussehen soll, müssen wir der Klasse Book ein paar Markierungen geben:
    - @Entity, @Table, eine ID und @GeneratedValue für die ID

```java
package com.raphaeladohrmann.booktracker.model;

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data                 // Generiert Getter, Setter, toString, equals/hashCode
@NoArgsConstructor    // Generiert leeren Konstruktor
@AllArgsConstructor   // Generiert Konstruktor mit allen Feldern
@Entity               // Markiert die Klasse als Datenbank-Tabelle
@Table(name = "books") // ist optional  und benennt die Tabelle explizit
public class Book {

    @Id // Primärschlüssel
    @GeneratedValue(strategy = GenerationType.IDENTITY) // Datenbank zählt ID automatisch hoch
    private Long id;
    private String title;
    private String author;
    private String isbn;
    private int rating;      // 1-5 Sterne
    private String status;   // z.B. "Gelesen", "Geplant"

}
```

13. **MySQL Datenbank anbinden**
    - Konfiguration: in den application.properties
    - In der Application Class die Annotation vom exclude befreien
    - Datenbank anlegen – booktracker
    - Application starten
14. **HTTP File anpassen**
    - ID entfernen -> wird automatisch generiert

Herzlichen Glückwunsch, wir haben einen kleinen Microservice, der eine GET und eine POST-Methode hat!

Erweiterungen:

15. **Application.properties sicher machen**
    - Ersetze die Werte für Username und Password und hinterlege sie direkt in IntelliJ. Willst du dein Projekt später irgendwo hosten, musst die die Werte dann da ebenfalls „hineingeben“.
    - Startkonfiguration anpassen und Environment Variables mitgeben
16. **In Git einchecken**
    - Gitignore anpassen! Gerne Selbstrecherche, was raus muss. Ein Beispiel ist sonst in meinem Repo.
17. **DELETE und UPDATE Endpunkt bauen**
    - Branch zum Nachschauen: delete_update_feature
18. **Weitere Recherche:**
    - Why Is Field Injection Not Recommended? | Baeldung